

DATA 266 — Assignment 2: Findings Report

Ritika Mukesh Neema | SJSU | Fall 2026

Part 1 — Prompt Engineering Techniques

Six prompting techniques were tested using LangChain with Google Gemini (gemini-1.5-flash), each with two distinct examples: one math/logic problem and one text-based task.

1.1 Zero-Shot

Approach: The model receives only the task — no examples, no reasoning instructions.

Findings: Effective for straightforward tasks (summing odd numbers, sentiment classification). However, on the sentiment example, the model gave a correct label but offered minimal justification. For simple, well-known problem types, zero-shot is sufficient and token-efficient.

1.2 Few-Shot

Approach: Several labeled input-output examples precede the actual question.

Findings: The model reliably matched the demonstrated pattern — correctly identifying the geometric sequence multiplier and producing the expected sentiment label with reasoning. Few-shot outperformed zero-shot on classification consistency because the examples anchored the output format. The cost is context-window tokens consumed by examples.

1.3 Chain-of-Thought (CoT)

Approach: Examples include explicit step-by-step reasoning traces.

Findings: On the arithmetic word problem, CoT produced a correct, verifiable solution with labeled intermediate steps ($60 \text{ mph} \times 2.5 \text{ h} = 150 \text{ miles}$). On the syllogism, the model correctly applied deductive logic across premises. CoT's strength is transparency: the visible reasoning chain makes errors easy to spot and debug.

1.4 Zero-Shot CoT

Approach: Append 'Let's think step by step' — no examples provided.

Findings: Surprisingly effective. On the rectangle area problem, the model set up simultaneous equations and solved correctly without any demonstration. On the scheduling task, it correctly chained time additions. Zero-Shot CoT is the best effort-to-quality ratio for moderate-difficulty reasoning tasks.

1.5 Meta-Prompting

Approach: Ask the model to first design its own solution strategy, then execute it.

Findings: On the fencing optimization problem, the model identified it as a constrained optimization, chose calculus/AM-GM inequality as the strategy, then executed correctly ($25\text{m} \times 25\text{m}$ square). On the fact-vs-opinion task, it first defined 'fact' and 'opinion' as categories, then systematically classified each sentence. Meta-prompting adds overhead but improves reliability on novel or ambiguous tasks where the 'right approach' is itself uncertain.

1.6 Tree of Thoughts (ToT)

Approach: Explore multiple reasoning paths, evaluate each, select the best.

Findings: On the water jug puzzle, the model explored three starting states, identified dead ends in two of them, and converged on the correct 7-step solution via Path 1. On the bakery revenue problem, it evaluated three strategies with explicit pros/cons and selected the online expansion strategy as having the highest probability of meeting the 20% target. ToT is the most token-expensive technique but the most robust for problems with non-obvious solution paths.

1.7 Comparative Summary

Technique	Accuracy	Verbosity	Best Use Case
Zero-Shot	Good (simple tasks)	Low	Well-defined, common tasks
Few-Shot	High	Medium	Classification, formatting
CoT	High	High	Multi-step math, logic
Zero-Shot CoT	Good–High	Medium	Quick reasoning boost
Meta-Prompting	High	High	Novel/ambiguous problems
Tree of Thoughts	Highest	Very High	Puzzles, planning, strategy

Key takeaway: There is a clear trade-off between output quality and token cost. For production systems, Zero-Shot CoT offers the best balance. For high-stakes reasoning, CoT with manual exemplars or ToT is worth the additional cost.

Part 2 — Self-Attention from Scratch

We implemented single-head scaled dot-product self-attention entirely from scratch using PyTorch primitives (nn.Embedding, nn.Linear, matrix multiplication, softmax). No transformer library classes were used.

2.1 Architecture

The model consists of four components: (1) a trainable word embedding layer ($d_{\text{model}} = 32$), (2) learned Q, K, V projection matrices, (3) the scaled dot-product attention function ($\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d_k})V$), and (4) a linear output head projecting to vocabulary logits. No positional encoding was used, so attention patterns are purely content-based.

2.2 Training Setup

Both models were trained for 500 epochs using Adam (lr=0.01) with cross-entropy loss on a next-token prediction objective. The dataset is a 5-sentence paragraph tokenized at word level (~37 tokens, ~30 unique words). The training converged well, with losses dropping below 1.0 for both variants.

2.3 Unmasked Self-Attention — Findings

The unmasked attention heatmap reveals several key patterns after training:

- Semantically related tokens develop higher mutual attention weights (e.g., 'models' and 'representations'; 'attention' and 'focus').
- Function words ('of', 'from', 'on') tend to receive less attention compared to content words.
- The attention distribution is non-uniform and clearly learned — distinct from the near-uniform weights seen at random initialization.
- Since no mask is applied, each token attends to both past and future positions, visible as non-zero values across the entire matrix.

2.4 Causal (Masked) Self-Attention — Findings

The causal mask enforces autoregressive attention by setting all entries above the diagonal to $-\infty$ before softmax, producing exactly zero attention weight for future positions:

- The upper triangle of the heatmap is uniformly zero, confirming that no token attends to any future position.
- The lower triangle shows learned attention patterns — the model must rely solely on past context to predict each next token.
- Early tokens (which have very little left context) show near-uniform attention over the few available positions, while later tokens develop sharper, more selective attention.
- The training loss is slightly higher than the unmasked model, which is expected since the masked model has strictly less information available at each position.

2.5 Unmasked vs. Masked — Comparative Analysis

Aspect	Unmasked	Causal (Masked)
Attention scope	Bidirectional — all tokens	Left-to-right only
Heatmap structure	Full matrix, non-zero everywhere	Lower-triangular; upper = 0

Training loss (final)	Lower (more context)	Higher (restricted context)
Attention sharpness	Sharp across all positions	Sharpens with more left context
Analogous architecture	BERT (encoder)	GPT (decoder)
Generation capability	Cannot generate autoregressively	Can generate token-by-token

Conclusion: The causal mask is essential for autoregressive language models. Without it, the model would 'leak' future information during training, making it unable to generate coherent text at inference time. Our from-scratch implementation confirms the theoretical properties described in Vaswani et al. (2017), Section 3.2: the attention weights form a valid probability distribution over allowed positions, and the causal mask correctly prevents information flow from future to past.